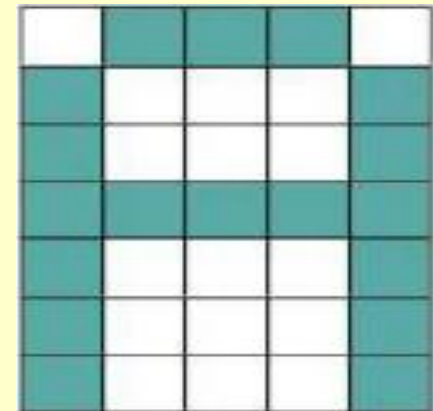


Chapitre N°6:

Python et traitement des images

Codage des images: Après le texte, l'image est le support le plus utilisé pour communiquer. Dans le domaine du numérique, les images sont constituées d'une matrice $L \times H$ (Largeur x Hauteur) de points élémentaires que l'on nomme généralement des pixels (abréviation de Picture Element). Chaque pixel a une couleur codée sur un nombre plus ou moins grand de bits. Le périphérique de sortie (écran, imprimante, ...) se doit de restituer les pixels de manière ordonnée en fonction de leur position respective (x, y) et de leur couleur.

Exemple : Le caractère ASCII « A » peut être simplement représenté sur une matrice 5×7 en « allumant » les pixels adéquats. La « couleur » peut-être ici matérialisée par un unique bit (pixel allumé ou éteint). Le codage d'une image peut donc dans un premier temps se résumer en la succession des codages des pixels suivant un ordre bien défini (balayage lignes-colonnes en partant du coin supérieur gauche).



On distingue trois types d'images :



Images en niveaux de gris
 $I(x,y) \in [0..255]$

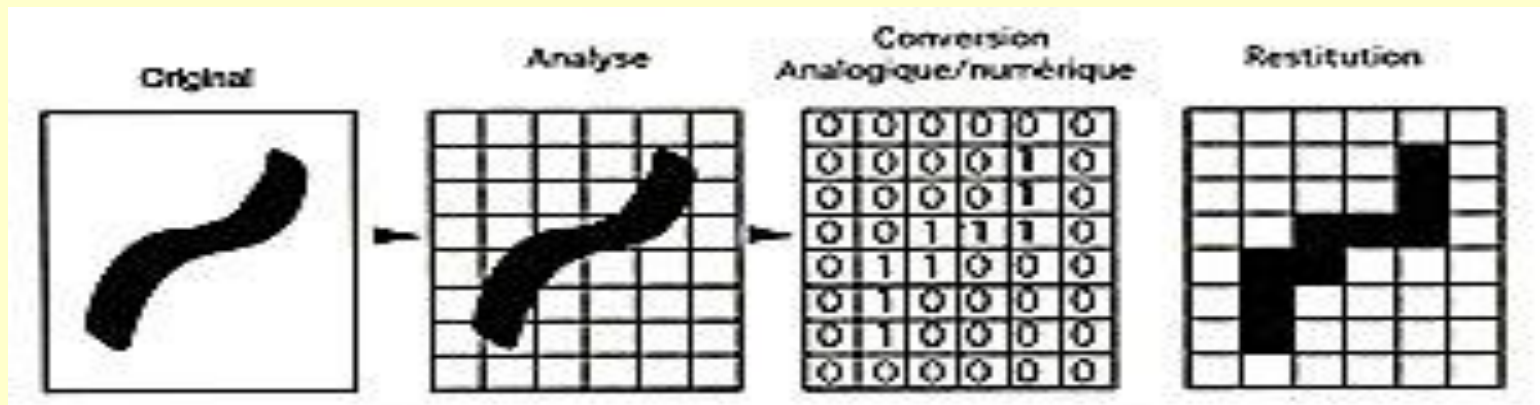


Images binaires
 $I(x,y) \in \{0, 1\}$



Images couleurs
 $I_R(x,y) \quad I_G(x,y) \quad I_B(x,y)$

Image noir et blanc : chaque pixel est codé sur 1bit



Images en niveau de gris : chaque pixel est codé sur 8bits=1octet

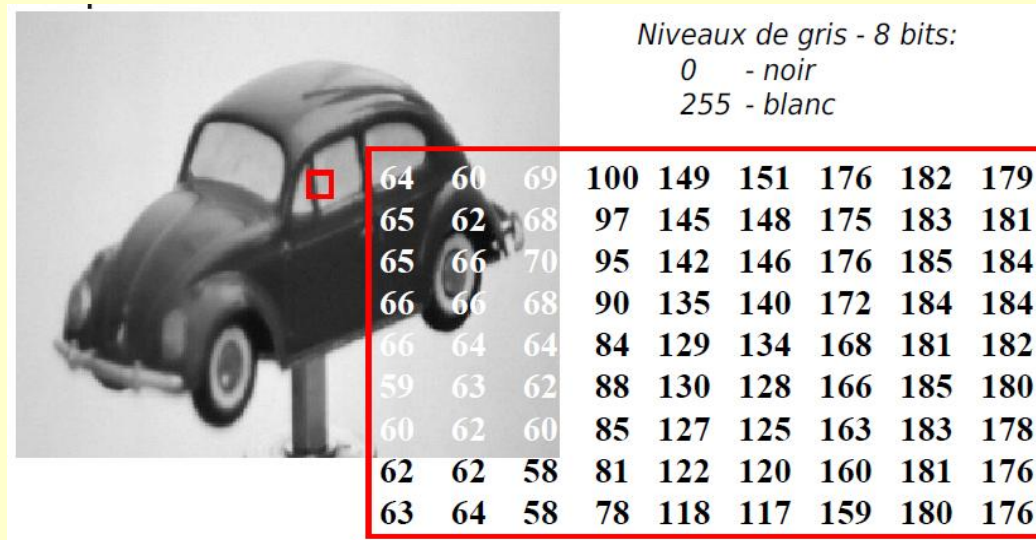
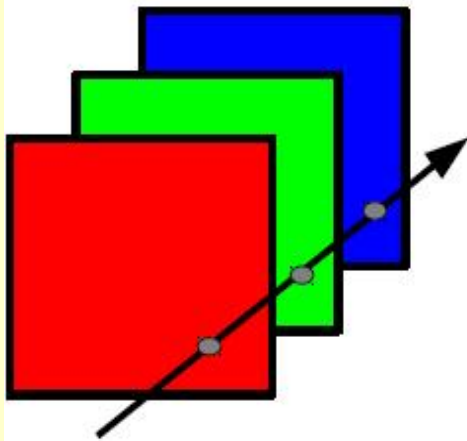
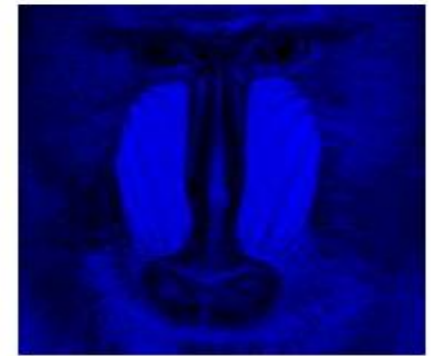


Image en couleur avec le mode RVB (Rouge, Vert et Bleu, ou en anglais RGB) correspond à celui fourni par la plupart des caméras couleur, il est naturellement utilisé pour la reproduction de couleurs sur écran (base noire). C'est le mode de composition des couleurs basé sur le principe des couleurs additives : le rouge, le vert et le bleu sont les trois primaires utilisés dans la constitution de couleurs à partir de sources lumineuses. Une image en couleur est codé sur 24 bits (Rouge=8bits, Vert=8bits, Bleu=8bits) chaque plan a les valeurs entre [0 et 255]

Exemple : (codage RVB)

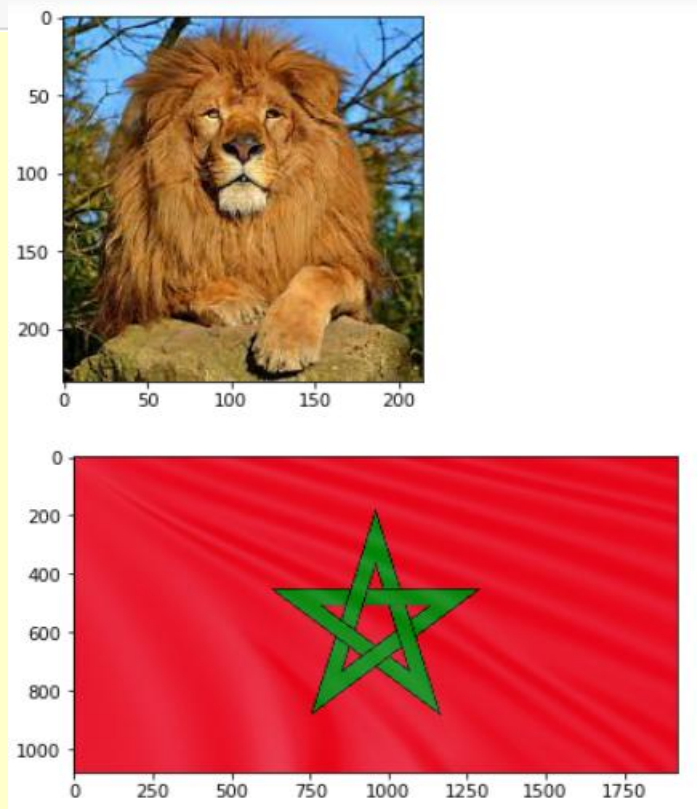


*Mais il existe aussi
d'autres codages de
la couleur que RVB...*

```

1 import matplotlib.pyplot as plt
2 import matplotlib.image as mpimg
3 import numpy as np
4 import os
5 os.chdir("E:\\informatique 5 septembre 2025\\chap 6 traitement images python\\")
6 img = mpimg.imread("lion.jpg")
7 img2=mpimg.imread("Maroc.jpg")
8 #img=np.flipud(img)
9 plt.imshow(img)
10 plt.show()
11 plt.imshow(img2)
12 plt.show()

```

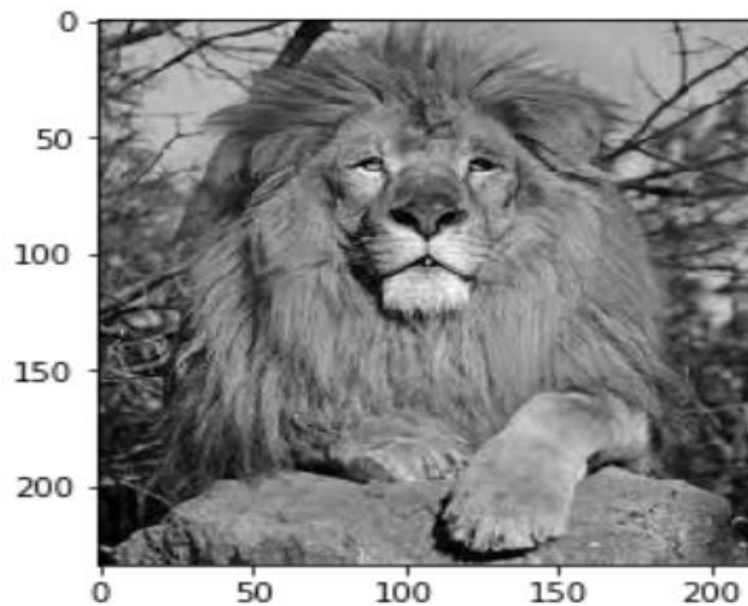


1	img
array([[[89, 116, 143],	
	[49, 83, 118],
	[52, 96, 143],
	...,
	[96, 180, 244],
	[93, 177, 241],
	[90, 174, 238]],
	[[88, 130, 168],
	[73, 120, 164],
	[65, 121, 178],
	...,
	[99, 182, 250],

```

1  rouge=img[:, :, 0]
2  vert=img[:, :, 1]
3  bleu=img[:, :, 2]
4  imgGris=0.2989*rouge+0.5870*vert+0.1140*bleu
5  plt.imshow(imgGris, cmap='gray')
6  plt.show()

```



```

1  print(rouge)

```

```

[[ 89  49  52 ...  96  93  90]
 [ 88  73  65 ...  99  97  94]
 [ 73  91  92 ... 104 102  99]

```

```
1 import matplotlib.pyplot as plt
2 import matplotlib.image as mpimg
3 import numpy as np
4 import os
5 os.chdir("E:\_____informatique")
6 img = mpimg.imread("lion.jpg")
7 img2=mpimg.imread("Maroc.jpg")
8 plt.subplot(1,2,1); plt.imshow(img); plt.title("Lion"); plt.axis('off')
9 plt.subplot(1,2,2); plt.imshow(img2); plt.title("Maroc"); plt.axis('off')
10 plt.show()
```

Lion



Maroc



```

1 os.chdir("E:\
2 import matplotlib.pyplot as plt
3 import matplotlib.image as mpimg
4 import numpy as np
5 img = mpimg.imread("lion.jpg")
6 img2=mpimg.imread("Maroc.png")
7
8 rouge=img[:, :,0]
9 vert=img[:, :,1]
10 bleu=img[:, :,2]
11
12 imgGris=0.2989*rouge+0.5870*vert+0.1140*bleu
13
14 Imnb=np.zeros(np.shape(imgGris))
15 seuil=np.mean(imgGris)
16 for i in range(np.shape(imgGris)[0]):
17     for j in range(np.shape(imgGris)[1]):
18         if imgGris[i,j]>=seuil:
19             Imnb[i,j]=1
20
21 plt.subplot(2,2,1); plt.imshow(img); plt.title("Lion"); plt.axis('off')
22 plt.subplot(2,2,2); plt.imshow(img2); plt.title("Maroc"); plt.axis('off')
23 plt.subplot(2,2,3); plt.imshow(imgGris,cmap='gray'); plt.title("Lion Gris"); plt.axis('off')
24 plt.subplot(2,2,4); plt.imshow(Imnb,cmap='gray'); plt.title("Lion en noir et blanc"); plt.axis('off')
25 plt.show()

```

Lion



Maroc



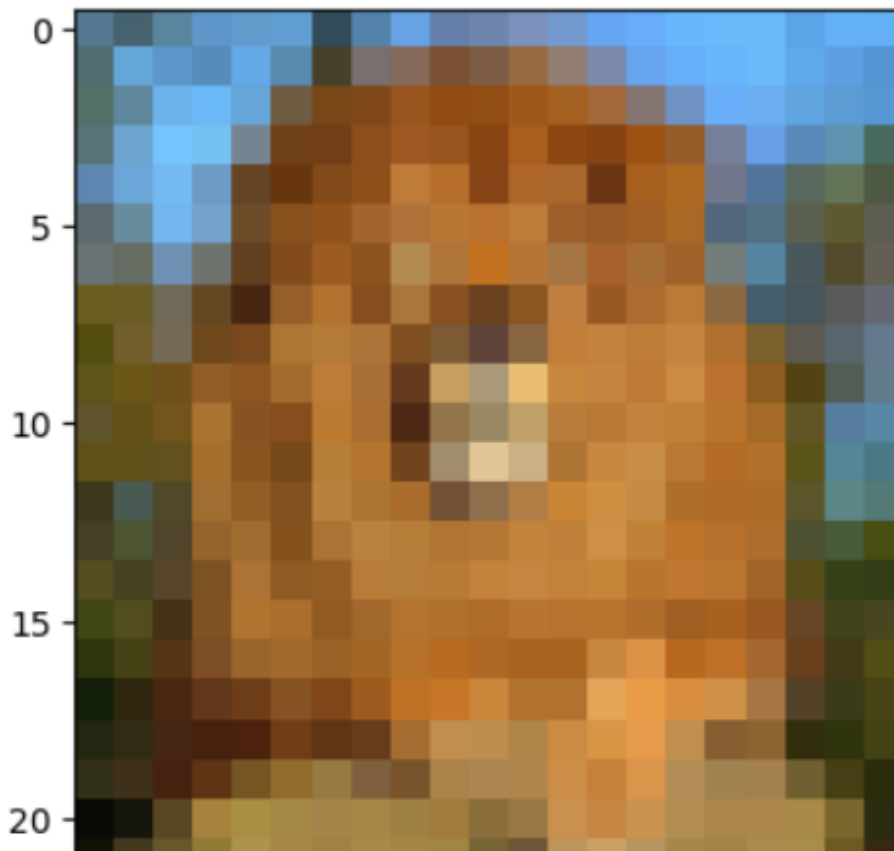
Lion Gris



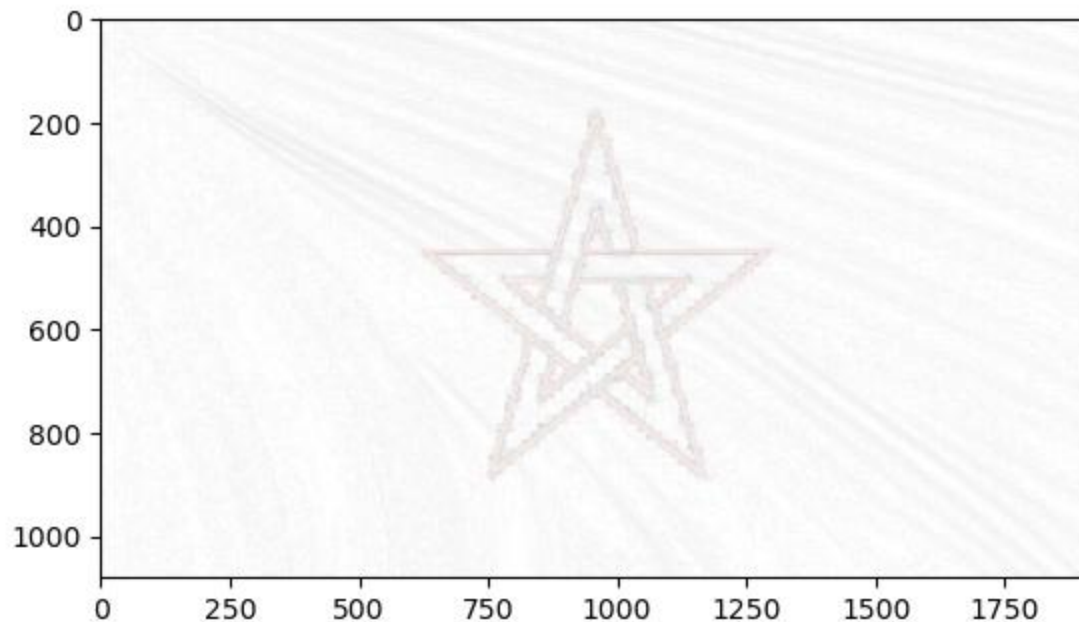
Lion en noir et blanc



```
1 import PIL.Image as image
2 img = image.open("lion.jpg")
3 img_size = img.size
4 nbLigne = int(img_size[0]/10)
5 nbColonne = int(img_size[1]/10)
6 res1 = img.resize((nbLigne,nbColonne))
7 plt.imshow(res1)
8 plt.show()
9 res2 = res1.resize((nbLigne*10,nbColonne*10))
10 plt.imshow(res2)
```



```
1 from scipy import ndimage
2 import matplotlib.image as mpimg
3 import matplotlib.pyplot as plt
4 import numpy as np
5 import os
6 os.chdir("E:\_____")
7 greyscale = mpimg.imread('Maroc.png')
8 edge_horizont = ndimage.sobel(greyscale, 1) |
9 edge_vertical = ndimage.sobel(greyscale, 0)
10 magnitude = np.hypot(edge_horizont, edge_vertical)
11 plt.imshow(magnitude, cmap='gray')
```



Les images en niveaux de gris sont représentées par une seule valeur d'intensité (entre 0 et 255 ou 0 et 1).
Pour les afficher, il faut une palette adaptée.

Sans cmap='gray', matplotlib peut afficher l'image en fausses couleurs (bleu, vert, rouge...), ce qui n'est pas naturel.

Donc on ajoute cmap='gray' pour :
✓ afficher correctement les images monochromes
✓ visualiser les contours, binarisation, filtres...
✓ éviter une coloration artificielle

La fonction color.rgb2gray() de la bibliothèque skimage convertit une image couleur en une image à un seul canal.
Elle utilise une combinaison pondérée des composantes rouges, vertes et bleues afin de respecter la perception humaine de la luminosité. Le paramètre principal est l'image RGB d'entrée.

```

21 import matplotlib.pyplot as plt
22 import matplotlib.image as mpimg
23 from skimage import color, filters, transform, exposure, util
24
25 img1 = mpimg.imread("lion.jpg")
26 img2 = mpimg.imread("FSBM.jpg")
27 gray1 = color.rgb2gray(img1)
28 gray2 = color.rgb2gray(img2)
29
30 plt.figure(figsize=(10,4))
31 plt.subplot(1,2,1); plt.imshow(gray1, cmap='gray'); plt.title("Lion - Gris"); plt.axis('off')
32 plt.subplot(1,2,2); plt.imshow(gray2, cmap='gray'); plt.title("FSBM - Gris"); plt.axis('off')
33 plt.show()

```

Lion - Gris



FSBM - Gris



#Le flou est appliqué avec `filters.gaussian()`, qui utilise une "fonction gaussienne" pour lisser l'image.

#Le paramètre le plus important est `sigma`, qui définit l'intensité du flou : plus `sigma` est grand, plus l'image est floue.

```
3 import numpy as np
4 from skimage import filters, transform, exposure, util
5 blur1 = filters.gaussian(img1, sigma=2)
6 blur2 = filters.gaussian(img2, sigma=2)
7
8 plt.figure(figsize=(10,4))
9 plt.subplot(2,2,1); plt.imshow(blur1); plt.title("Lion - Flou"); plt.axis('off')
10 plt.subplot(2,2,2); plt.imshow(blur2); plt.title("FSBM - Flou"); plt.axis('off')
11 plt.subplot(2,2,3); plt.imshow(img1); plt.title("Lion - Origine"); plt.axis('off')
12 plt.subplot(2,2,4); plt.imshow(img2); plt.title("FSBM - Origine"); plt.axis('off')
13 plt.show()
```

Lion - Flou



Lion - Origine



FSBM - Flou



FSBM - Origine



```

1 # Détection des contours avec le filtre Sobel.
2 # Ce filtre calcule les variations d'intensité entre les pixels pour mettre en évidence les bords.
3 edges1 = filters.sobel(gray1)
4 edges2 = filters.sobel(gray2)
5
6 plt.figure(figsize=(10,4))
7 plt.subplot(1,2,1); plt.imshow(edges1, cmap='gray'); plt.title("Lion - Contours"); plt.axis('off')
8 plt.subplot(1,2,2); plt.imshow(edges2, cmap='gray'); plt.title("FSBM - Contours"); plt.axis('off')
9 plt.show()
10

```

Lion - Contours



FSBM - Contours



```

1 # Rotation des images avec transform.rotate().
2 # Le paramètre principal est l'angle (en degrés). Ici, 45°.
3 import PIL.Image as image
4 img1 = image.open("lion.jpg") ; img2 = image.open("FSBM.jpg")
5 rot1 = img1.rotate(45)          ; rot2 = img2.rotate(-45)
6 rot3 = img1.rotate(45,expand=True)
7 rot4 = img2.rotate(-45,expand=True)
8 plt.figure(figsize=(10,4))
9 #plt.subplot(nrows, ncols, index) permet de diviser la figure en une grille et de placer plusieurs
10 #graphiques ou images dans cette grille.
11 #Paramètres: #nrows : nombre de lignes dans la grille
12 #ncols : nombre de colonnes dans la grille
13 #index : position actuelle (commence à 1, de gauche à droite, de haut en bas)
14 plt.subplot(2,2,1); plt.imshow(rot1); plt.title("Lion : Rot 45°"); plt.axis('off')
15 plt.subplot(2,2,2); plt.imshow(rot2); plt.title("FSBM : Rot -45°"); plt.axis('off')
16 plt.subplot(2,2,3); plt.imshow(rot3); plt.title("Lion : Rot 45°, expand=True"); plt.axis('off')
17 plt.subplot(2,2,4); plt.imshow(rot4); plt.title("FSBM : Rot -45°, expand=True"); plt.axis('off')
18 plt.show()

```

Lion : Rot 45°



Lion : Rot 45°, expand=True



FSBM : Rot -45°



FSBM : Rot -45°, expand=True



```

1  # Inversion des couleurs avec util.invert() (négatif photo).
2  # Chaque pixel est remplacé par son opposé : 1 - valeur ou 255 - valeur selon le format.
3  from skimage import util
4  from skimage import color
5  img1 = mpimg.imread("lion.jpg") ; img2 = mpimg.imread("FSBM.jpg")
6  inv1 = util.invert(img1)          ; inv2 = util.invert(img2)
7  gray1 = color.rgb2gray(img1); invGray1=util.invert(gray1)
8  gray2 = color.rgb2gray(img2); invGray2=util.invert(gray2)
9  plt.figure(figsize=(12,6))
10 plt.subplot(4,2,1); plt.imshow(inv1); plt.title("Lion - Inversé"); plt.axis('off')
11 plt.subplot(4,2,2); plt.imshow(inv2); plt.title("FSBM - Inversé"); plt.axis('off')
12 plt.subplot(4,2,3); plt.imshow(img1); plt.title("Lion - Origine"); plt.axis('off')
13 plt.subplot(4,2,4); plt.imshow(img2); plt.title("FSBM - Origine"); plt.axis('off')
14 plt.subplot(4,2,5); plt.imshow(gray1,cmap='gray'); plt.title("Lion - Gris"); plt.axis('off')
15 plt.subplot(4,2,6); plt.imshow(gray2,cmap='gray'); plt.title("FSBM - Gris"); plt.axis('off')
16 plt.subplot(4,2,7); plt.imshow(invGray1,cmap='gray'); plt.title("Lion - Gris - inversé"); plt.axis('off')
17 plt.subplot(4,2,8); plt.imshow(invGray2,cmap='gray'); plt.title("FSBM - Gris-inversé"); plt.axis('off')
18 plt.show()

```

Lion - Inversé



Lion - Origine



Lion - Gris



Lion - Gris - inversé



FSBM - Inversé



FSBM - Origine



FSBM - Gris



FSBM - Gris-inversé



Augmentation du contraste avec `equalize_hist()`.

L'algorithme redistribue les niveaux de gris pour améliorer les zones sombres ou trop claires.

```
3 contr1 = exposure.equalize_hist(img1)
4 gray1 = color.rgb2gray(img1); contrasteGray1=exposure.equalize_hist(gray1)
5 plt.subplot(2,2,1); plt.imshow(img1); plt.title("Lion - Origine"); plt.axis('off')
6 plt.subplot(2,2,2); plt.imshow(contr1); plt.title("Lion - contraste"); plt.axis('off')
7 plt.subplot(2,2,3); plt.imshow(gray1,cmap='gray'); plt.title("Lion - Gris- Origine"); plt.axis('off')
8 plt.subplot(2,2,4); plt.imshow(contrasteGray1,cmap='gray'); plt.title("Lion - Gris-contraste"); plt.axis('off')
9 plt.show()
```

Lion - Origine



Lion - contraste



Lion - Gris- Origine



Lion - Gris-contraste



```
1 # Redimensionnement avec transform.resize().
2 # On donne la nouvelle taille (hauteur, largeur) en pixels. L'image est réinterpolée.
3 small1 = transform.resize(img1, (200, 200))
4 small2 = transform.resize(img2, (200, 200))
5 plt.subplot(1,2,1); plt.imshow(small1); plt.title("Lion - Réduit"); plt.axis('off')
6 plt.subplot(1,2,2); plt.imshow(small2); plt.title("FSBM - Réduit"); plt.axis('off')
7 plt.show()
```

Lion - Réduit



FSBM - Réduit



```

1 # Retourne l'image horizontalement (effet miroir).
2 # fliplr() inverse l'ordre des colonnes du tableau d'images.
3 flip1 = np.fliplr(img1) ; flip2 = np.fliplr(img2)
4 flip1 = np.fliplr(img1) ; flip2 = np.fliplr(img2)
5 plt.subplot(2,2,1); plt.imshow(img1); plt.title("Lion - Origine"); plt.axis('off')
6 plt.subplot(2,2,2); plt.imshow(img2); plt.title("FSBM - Origine"); plt.axis('off')
7 plt.subplot(2,2,3); plt.imshow(flip1); plt.title("Lion - Symétrie"); plt.axis('off')
8 plt.subplot(2,2,4); plt.imshow(flip2); plt.title("FSBM - Symétrie"); plt.axis('off')
9 plt.show()

```

Lion - Origine



Lion - Symétrie



FSBM - Origine



FSBM - Symétrie

